Universidade Estadual de Santa Cruz (UESC)

Etapa 2 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação paralela em memória distribuída com MPI.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Alecsander Almeida, Samuel Santos

Disciplina: DEC107 — Processamento Paralelo

Data: 29/10/2026

1. Introdução

Este relatório apresenta a segunda etapa do projeto do conjunto de Mandelbrot, que foca em implementar uma versão paralela em memória distribuída utilizando MPI (Message Passing Interface). O objetivo é avaliar como diferentes estratégias de distribuição das linhas da imagem entre processos afetam o desempenho, o balanceamento de carga e o custo de comunicação, além de comparar os resultados com a solução em OpenMP da etapa anterior.

O conjunto de Mandelbrot é definido pelos números complexos c tais que a recorrência $z_{n+1} = z_n^2 + c$ permaneça limitada quando iniciada em $z_0 = 0$. Na prática, calcula-se para cada pixel da imagem, mapeado para um ponto c do plano complexo, o número de iterações necessárias até que o módulo de z ultrapasse o raio de escape 2, respeitando um número máximo de iterações (MAX_ITER). Neste trabalho, MAX_ITER vale 1000 para a imagem

completa e 5000 para o caso de estresse do vale dos cavalos-marinhos. Esse número de iterações, o escape time, é o valor armazenado em cada pixel.

Diferente do OpenMP, em que as threads compartilham o mesmo espaço de memória, no paradigma de memória distribuída cada processo possui sua própria memória privada, e qualquer troca de dados deve ser feita explicitamente por troca de mensagens. O MPI é a interface padrão para esse modelo: os processos executam o mesmo programa, são identificados por um rank e se comunicam por meio de operações como envio e recebimento de mensagens ou comunicações coletivas, como o MPI_Gather. Nesse contexto, o primeiro passo da paralelização é a decomposição do domínio, isto é, decidir quais linhas da imagem cada processo calcula. Uma distribuição estática define essa divisão antes da execução, podendo ser em faixas contíguas de linhas (blocos) ou intercalada (cíclica). Uma distribuição dinâmica, como o esquema mestre-trabalhador, entrega linhas aos processos sob demanda durante a execução. Nesta etapa foram implementadas duas estratégias estáticas: a decomposição em blocos contíguos e a distribuição cíclica de linhas.

O Mandelbrot é um problema embaraçosamente paralelo, já que o cálculo de cada pixel é independente dos demais, mas seu custo é fortemente desbalanceado: pixels fora do conjunto escapam em poucas iterações, enquanto pixels dentro do conjunto ou próximos à sua fronteira consomem todas as MAX_ITER iterações. Em memória distribuída, esse desbalanceamento tem um peso maior do que em memória compartilhada. No OpenMP, o escalonamento dinâmico permite redistribuir linhas entre as threads durante a execução com um custo baixo. Em MPI, uma distribuição estática fixa o trabalho de cada processo antes de o cálculo começar e não há como corrigi-la depois. Se a divisão atribuir a um processo uma região mais custosa da imagem, os demais terminarão antes e ficarão ociosos esperando por ele. Como o conjunto é simétrico em relação ao eixo real e as linhas mais custosas se concentram na região central da imagem, a expectativa é que a decomposição em blocos sobrecarregue os processos que recebem as faixas centrais, enquanto a distribuição cíclica, ao intercalar as linhas, espalhe esse custo de forma mais uniforme. Por isso, a escolha da estratégia de distribuição é determinante para o desempenho, e é o que esta etapa investiga.

2. Metodologia

Implementação paralela com MPI. A versão paralela foi implementada em C com MPI, e todos os processos executam o mesmo programa (modelo SPMD), sendo diferenciados pelo rank, obtido com MPI_Comm_rank, e pelo número total de processos, obtido com MPI_Comm_size. As funções utilizadas foram:

- MPI_Init e MPI_Finalize: inicializam e encerram o ambiente MPI.
- MPI_Barrier: sincroniza todos os processos antes de cada medição de tempo, para que todos comecem a contar do mesmo ponto.
- MPI_Wtime: mede o tempo.
- MPI_Gather: coleta, no rank 0, os blocos de linhas calculados por cada processo, formando a matriz final, e também os tempos de computação e de comunicação de cada processo. Como size é divisível por nprocs, todos os blocos têm o mesmo tamanho, e por isso MPI_Gatherv não foi necessário, no entanto isso impediu de realizar testes .
- MPI_Abort: encerra todos os processos em caso de falha de alocação de memória.

A função de cálculo de uma linha (calc_row) segue a mesma lógica da versão serial: mapeia o pixel para o plano complexo, itera a recorrência com as partes real e imaginária em variáveis double separadas e usa o critério de escape $z_real^2 + z_imaginary^2 > 4$. O resultado de cada pixel é armazenado em int32_t, formato pedido no trabalho.

Sobre a estratégia de distribuição, ela foi implementada em duas estratégias estáticas, ambas com granularidade de uma linha por unidade de trabalho e com n = size / nprocs linhas por processo:

- static (blocos contíguos): o processo de rank r calcula as linhas de r*n até r*n + n - 1. Cada processo recebe uma faixa contínua da imagem, e o MPI_Gather já entrega os blocos na ordem correta.
- cyclic (cíclica): o processo de rank r calcula as linhas r, r + P, r + 2P, e assim por diante, sendo P o número de processos. Como o MPI_Gather concatena os blocos por rank, o rank 0 precisa reordenar as linhas recebidas até a posição correta na imagem (função reorder_cyclic, que copia linha a linha com memcpy), o que exige um segundo buffer do tamanho da imagem.

Em ambas, todos os processos calculam e enviam seus resultados. O rank 0 também recebe a imagem, reordena (no cíclico) e grava o arquivo de saída. Não foi implementada distribuição dinâmica (mestre-trabalhador). Uma limitação das duas estratégias é que exigem size divisíveis por nprocs, o que restringe o experimento a 1, 2, 4, 8 e 16 processos.

Além disso, cada processo calcula suas linhas em um buffer local de n * size inteiros. A coleta é feita com um único MPI_Gather para o rank 0, que monta a matriz completa de size × size inteiros de 32 bits em ordem row-major. A escrita do arquivo binário é feita somente pelo rank 0, após a coleta, e seu tempo é medido e reportado separadamente. Não foi utilizado nenhum tipo de MPI_Broadcast, MPI_Send ou MPI_Recv.

[Descrição do hardware/ambiente de execução utilizado nos testes: CPU, número de cores, memória, sistema operacional, compilador, implementação e versão do MPI, parâmetros de execução (ex.: mpirun -np N, binding)];

Todas as medições foram feitas com MPI_Wtime. Antes de iniciar o cálculo, os processos são sincronizados com MPI_Barrier. Cada processo mede o próprio tempo de computação, que cobre apenas o laço de cálculo de suas linhas. Em seguida, os processos são sincronizados novamente e cada um mede o tempo da chamada MPI_Gather que coleta a imagem. O tempo de escrita em disco também é medido separadamente. Os tempos de computação e de comunicação de cada processo são reunidos no rank 0 por meio de MPI_Gather. O tempo total de geração da imagem foi definido como T_max (maior tempo de computação entre os processos) somado ao tempo de coleta no rank 0. Como o tempo de comunicação dos processos que apenas enviam pode terminar antes da conclusão da coleta, o custo de coleta da imagem foi considerado como o tempo medido pelo rank 0. Para cada configuração foram realizados 72 testes, e a métrica utilizada foi a mediana.

As métricas de avaliação utilizadas foram:

- Tempo de execução: tempo total de geração da imagem, conforme definido acima, em segundos.
- Speedup: razão entre o tempo da versão de referência e o tempo da versão paralela com P processos: Speedup(P) = T_ref / T_paralelo(P). [Definir a referência utilizada: versão serial compilada com o mesmo compilador e flags, ou execução MPI com P = 1.]
- Eficiência: razão entre o Speedup e o número de processos: Eficiência(P) = Speedup(P) / P.
- Fator de Balanceamento de Carga (FBC): FBC = (T_max − T_min) / T_max, em que T_max e T_min são o maior e o menor tempo de computação entre os processos MPI.

Nesse cálculo, T considera apenas o tempo de computação de cada processo, medido com MPI_Wtime, sem incluir a comunicação. FBC igual a 0 indica balanceamento perfeito, e valores próximos de 1 indicam forte desbalanceamento. Esta definição é diferente do fator carga máxima / carga média, baseado em iterações, usado na Etapa 1, e por isso os valores das duas etapas não são diretamente comparáveis.

- Escalabilidade forte: a resolução é mantida em 4096 × 4096 e o número de processos é variado, para avaliar a redução do tempo com mais recursos.
- Escalabilidade fraca: o trabalho por processo é mantido aproximadamente constante, aumentando a resolução junto com o número de processos (por exemplo, 4096², 8192² e 16384²).
- [Definição do Fator de Balanceamento de Carga (FBC), conforme adotado em sala de aula:] FBC = (T_max − T_min) / T_max [onde T_max e T_min são, respectivamente, o maior e o menor tempo de computação entre os processos MPI. Indicar explicitamente o que foi considerado em T (apenas computação ou computação + comunicação) e como foi medido. Interpretação: FBC = 0 indica balanceamento perfeito; valores próximos de 1 indicam forte desbalanceamento];

3. Resultados

Para verificar a corretude do algoritmo, foi realizada uma comparação entre os resultados obtidos pela versão paralelizada e aqueles produzidos pelo código sequencial, analisando-se cada resultado individualmente, de maneira análoga ao procedimento adotado na implementação com OpenMPI.

case	strategy	size	processes	different_pixels	max_abs_difference	allowed_different_pixels	allowed_max_abs_difference	status
standard	static	4096	1	0	0	1677	1	APPROVED_EXACT
standard	static	4096	2	0	0	1677	1	APPROVED_EXACT
standard	static	4096	4	0	0	1677	1	APPROVED_EXACT

standard	static	4096	8	0	0	1677	1	APPROVED_EXACT
standard	static	4096	16	0	0	1677	1	APPROVED_EXACT
standard	cyclic	4096	1	0	0	1677	1	APPROVED_EXACT
standard	cyclic	4096	2	0	0	1677	1	APPROVED_EXACT
standard	cyclic	4096	4	0	0	1677	1	APPROVED_EXACT
standard	cyclic	4096	8	0	0	1677	1	APPROVED_EXACT
standard	cyclic	4096	16	0	0	1677	1	APPROVED_EXACT
seahorse	static	4096	16	0	0	1677	1	APPROVED_EXACT
seahorse	cyclic	4096	16	0	0	1677	1	APPROVED_EXACT

Tabela 1: Corretude

A Tabela 1 apresenta os resultados dessa comparação, evidenciando a ausência de divergências entre as execuções nas 12 configurações avaliadas. Não foram encontradas grandes diferenças nos dados que justifiquem uma quebra de corretude. Dessa forma, os resultados obtidos confirmam a corretude da implementação paralela utilizando MPI.

strategy	processes	serial_median_s	total_median_s	compute_median_s	gather_median_s	master_overhead_median_s	speedup	efficiency	efficiency_percent
cyclic	1	6.402.149.916	6.448.042.900	6.418.287.400	12.076.400	15.503.500	992.883	992.883	99.288
cyclic	2	6.402.149.916	3.273.457.100	3.233.166.600	12.372.200	27.545.800	1.955.776	977.888	97.789
cyclic	4	6.402.149.916	1.677.856.100	1.636.005.500	15.433.800	25.808.100	3.815.673	953.918	95.392

cyclic	8	6.402.149.916	914.094.900	864.340.600	22.034.500	27.719.800	7.003.813	875.477	87.548
cyclic	16	6.402.149.916	551.920.700	502.073.500	27.638.300	22.513.800	11.599.764	724.985	72.499
static	1	6.402.149.916	6.399.254.000	6.390.414.000	8.840.000	0	1.000.453	1.000.453	100.045
static	2	6.402.149.916	3.203.948.100	3.192.231.300	11.216.300	0	1.998.206	999.103	99.910
static	4	6.402.149.916	3.174.371.300	3.159.164.000	16.029.000	0	2.016.825	504.206	50.421
static	8	6.402.149.916	2.204.418.100	2.183.869.700	20.548.400	0	2.904.236	363.029	36.303
static	16	6.402.149.916	1.302.712.000	1.277.332.500	26.630.100	0	4.914.478	307.155	30.715

Tabela 2: Tempos obtidos para o input padrão em variadas estratégias

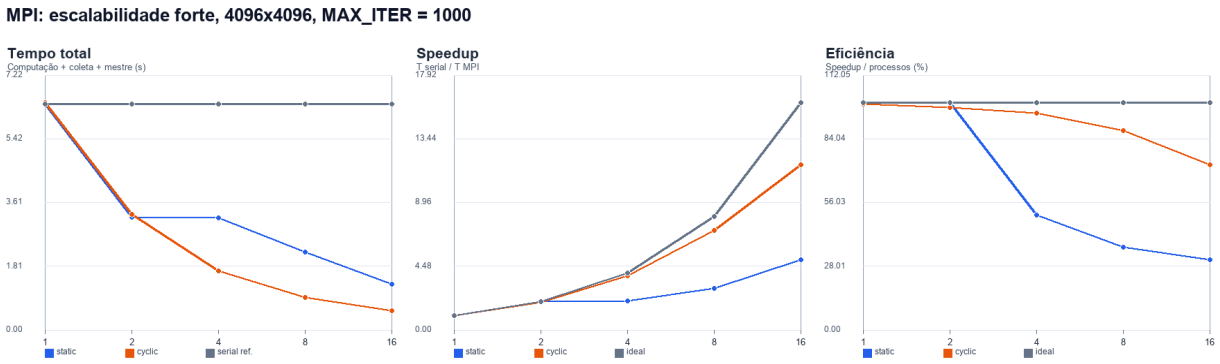


Imagem 1: Escalabilidade forte com input padrão

A análise do *Speedup* demonstrou que a distribuição *static* apresentou uma tendência de estagnação dos ganhos de desempenho e de redução da eficiência à medida que o número de processos aumentou. Em contrapartida, a distribuição *cyclic* apresentou um crescimento considerável do *Speedup*, mantendo-se próxima ao comportamento ideal. A partir desses resultados, é possível inferir que a distribuição *static* apresenta limitações de escalabilidade, enquanto a distribuição *cyclic* demonstra melhor

capacidade de aproveitamento do aumento do número de processos, mostrando-se mais adequada às configurações avaliadas.

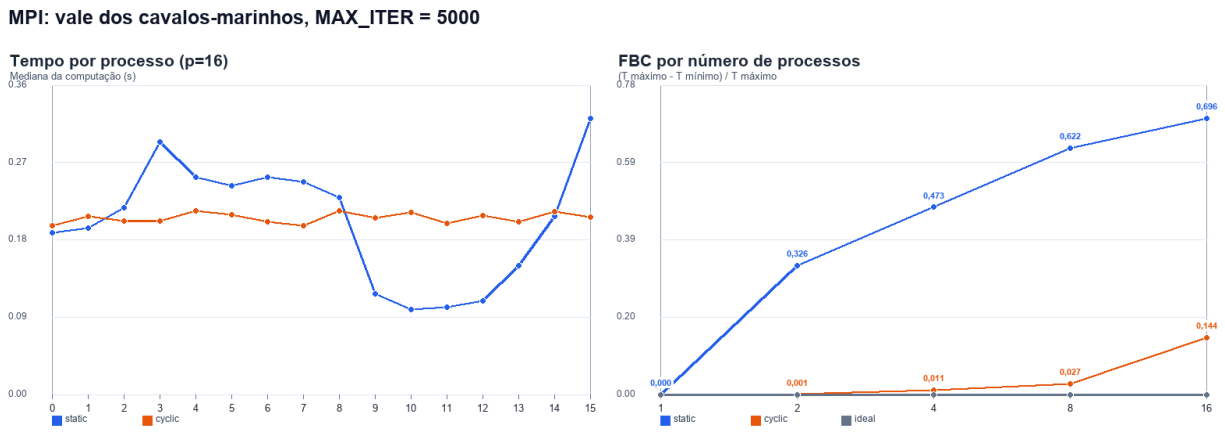


Imagem 2: Tempo de Computação e FBC

A estratégia cyclic manteve baixos valores de FBC com o aumento do número de processos, indicando bom equilíbrio de carga, embora tenha apresentado uma elevação com 16 processos. Já a estratégia static exibiu crescimento mais apressado do FBC, passando de 0,3255 com 2 processos para 0,6965 com 16, o que evidencia um desbalanceamento de carga cada vez maior.

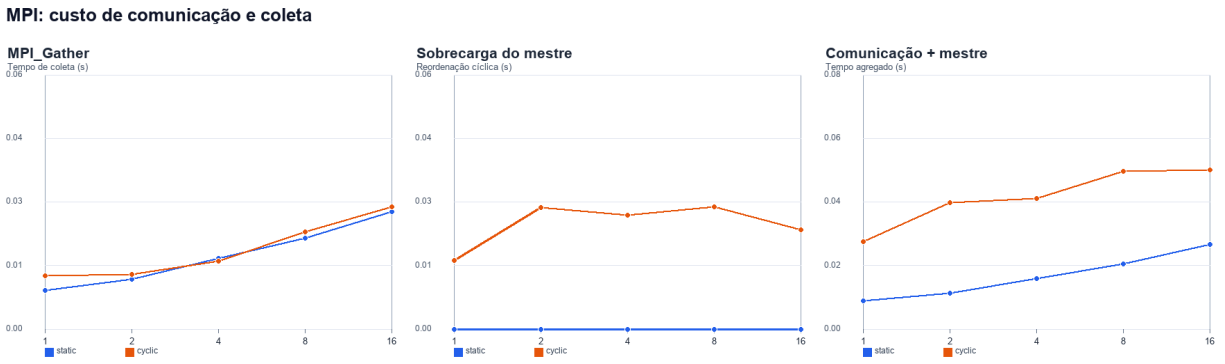


Imagem 3: Custo de comunicação e coleta

A operação MPI_Gather é responsável por reunir os blocos da imagem calculados pelos processos no processo mestre (rank 0). Nas estratégias static e cyclic, a quantidade de dados transferida é aproximadamente a mesma, visto que cada processo calcula e envia a mesma quantidade de linhas. Dessa forma, o tempo do Gather se mantém semelhante entre as estratégias, sendo as pequenas diferenças possivelmente relacionadas às variações experimentais, à sincronização e ao comportamento da memória.

A principal diferença ocorre após a coleta dos dados. No static, cada processo calcula um conjunto contínuo de linhas da imagem, permitindo que o Gather reúna os blocos diretamente na ordem correta, sem a necessidade de uma organização adicional. No entanto, no cyclic, os processos calculam linhas intercaladas da imagem, sendo necessário que o processo mestre reorganize os dados coletados para suas posições corretas. Essa operação representa uma sobrecarga adicional para o mestre

Com 16 processos no input padrão, o static apresentou um tempo de 0,026630 s no Gather, sem sobrecarga adicional. Enquanto isso, o cyclic apresentou 0,027638 s no Gather e 0,022514 s na reorganização dos dados, totalizando 0,050152 s. A partir disso, é possível observar que o cyclic apresentou um custo maior na coleta e organização dos dados. Apesar disso, essa estratégia manteve um desempenho geral superior ao static, assim mostrando que a redução do desbalanceamento de carga compensou a sobrecarga adicional de reorganização.

Na Imagem 4, é possível analisar a escalabilidade forte das duas abordagens em comparação com o algoritmo sequencial. E na Imagem 5 apresenta a análise de escalabilidade fraca, considerando os mesmos parâmetros.

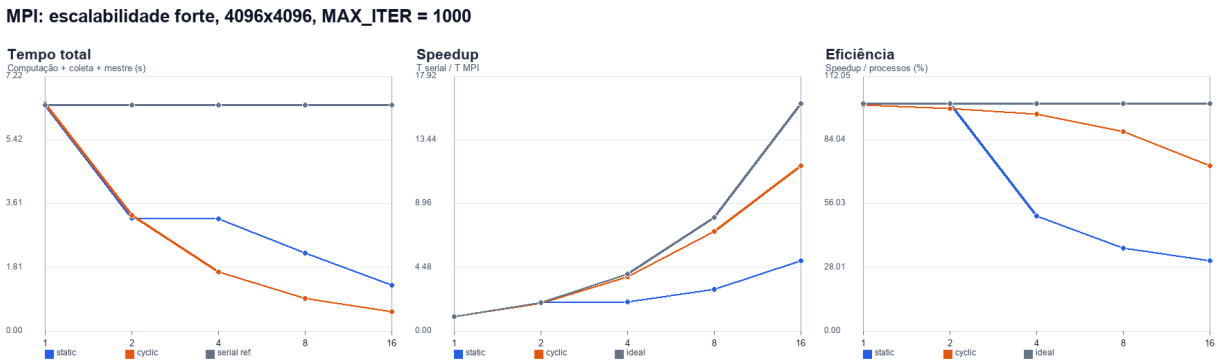


Imagem 4: Escalabilidade forte tempo total, Speedup e Eficiência para o Input Padrão em diferentes estratégias

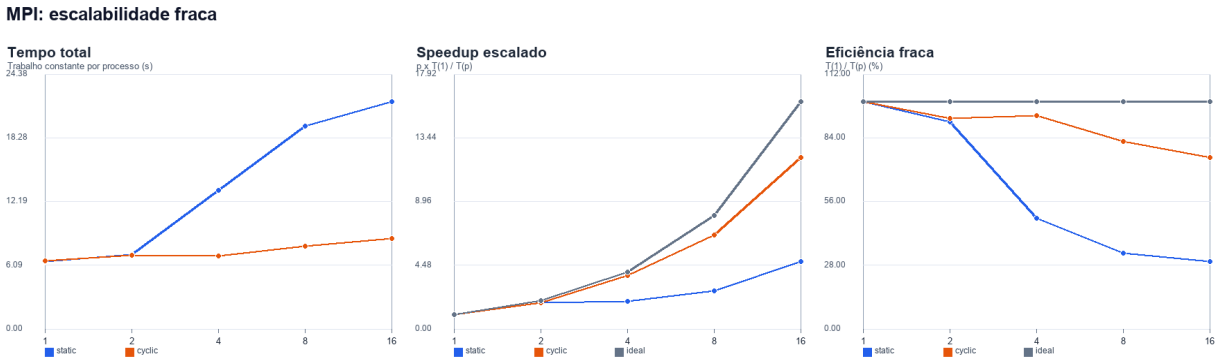


Imagem 5: Escalabilidade fraca tempo total, Speedup e Eficiência em diferentes estratégias

Para o comparativo entre as versões de OpenMP X MPI em 16 recursos de processamento, a solução OpenMP apresentou tempo de 1,266 s, Speedup de 5,015× e Eficiência de 31,3%. O MPI com distribuição static obteve resultados semelhantes: 1,303 s, Speedup de 4,914× e Eficiência de 30,7%. Já o MPI cyclic apresentou o melhor desempenho, com tempo de 0,552 s, Speedup de 11,600× e Eficiência de 72,5%. É possível ver isto na Imagem 5.

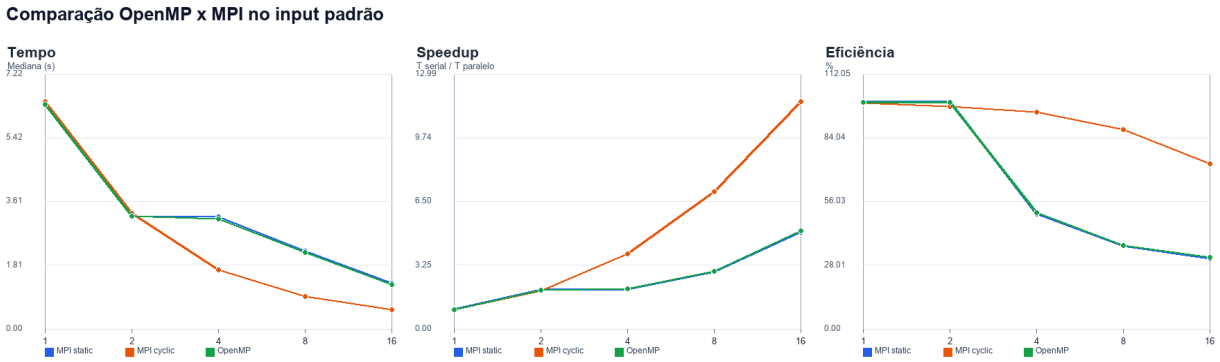


Imagem 6: Comparação com OpenMP

4. Discussão

Os resultados apresentados na tabela demonstraram que a implementação paralela com MPI produziu resultados equivalentes aos do algoritmo sequencial em todas as configurações avaliadas. Dessa forma, é possível afirmar que a implementação paralela com MPI apresentou corretude nos testes realizados.

Na comparação com a versão sequencial, foi possível observar uma superioridade de desempenho da implementação com MPI, influenciada por diferentes fatores, mas principalmente pela redução do desbalanceamento de carga. No entanto, não é adequado afirmar que o MPI seja superior ao OpenMP em todos os casos, visto que o desempenho depende das características do problema e das estratégias de paralelização utilizadas. Apesar disso, o MPI se mostrou uma abordagem adequada e eficiente para o cálculo do conjunto de Mandelbrot estratégia static apresentou uma implementação mais simples, visto que cada processo calcula um conjunto contínuo de linhas da imagem, permitindo que o MPI_Gather reúna os dados diretamente na ordem correta. No entanto, essa estratégia apresentou um maior desbalanceamento de carga no vale dos cavalos-marinhos, com FBC de 0,696498. No input padrão, com 16 processos, apresentou um tempo de 1,303 s e Speedup de 4,914×. Além disso, seu custo de comunicação foi menor, aproximadamente 0,027 s, sem a necessidade de reorganização dos dados.

Em contrapartida, a estratégia cyclic apresentou uma implementação mais complexa, pois distribui as linhas de forma intercalada entre os processos, sendo necessário reorganizar os

dados após o Gather. Com 16 processos, o custo de coleta e reorganização foi de aproximadamente 0,050 s. Apesar disso, apresentou um FBC de 0,143594, tempo de execução de 0,552 s e Speedup de 11,600×.

A partir desses resultados, é possível observar que o cyclic se mostrou mais adequado para o cálculo do conjunto de Mandelbrot, visto que a redução do desbalanceamento de carga compensou o custo adicional de comunicação e reorganização dos dados. Entretanto, em problemas com cargas computacionais mais uniformes, o static pode apresentar vantagens devido à sua implementação mais simples e menor sobrecarga.ot nas configurações avaliadas.

O aumento do número de processos permitiu reduzir o tempo de execução no escalonamento forte. No entanto, os ganhos de desempenho apresentaram uma tendência de redução devido a fatores como o desbalanceamento de carga, os custos de comunicação e as etapas seriais, conforme previsto pela Lei de Amdahl. Com 16 processos, a estratégia `cyclic` apresentou um *Speedup* de 11,600× e eficiência de 72,5%, enquanto a estratégia `static` obteve um *Speedup* de 4,914× e eficiência de 30,7%, demonstrando uma diferença considerável no aproveitamento dos processos.

Na escalabilidade fraca, a análise pela Lei de Gustafson demonstrou que o aumento do tamanho do problema, acompanhado pelo aumento do número de processos, permite um melhor aproveitamento dos recursos computacionais. Nesse cenário, o `cyclic` alcançou um *Speedup* escalado de 12,037× com 16 processos, enquanto o `static` apresentou apenas 4,736×. A partir desses resultados, é possível observar que o `cyclic` apresentou melhor escalabilidade nas configurações avaliadas. Apesar disso, ambas as estratégias permaneceram abaixo do desempenho ideal, principalmente devido aos custos de comunicação e ao desbalanceamento de carga.

O aumento do número de processos permitiu reduzir o tempo de execução no escalonamento forte. No entanto, os ganhos de desempenho apresentaram uma tendência de redução devido a fatores como o desbalanceamento de carga, os custos de comunicação e as etapas seriais, conforme previsto pela Lei de Amdahl. Com 16 processos, a estratégia cyclic apresentou um Speedup de 11,600× e eficiência de 72,5%, enquanto a estratégia static obteve um Speedup de 4,914× e eficiência de 30,7%, demonstrando uma diferença considerável no aproveitamento dos processos.

Na escalabilidade fraca, a análise pela Lei de Gustafson demonstrou que o aumento do tamanho do problema, acompanhado pelo aumento do número de processos, permite um melhor aproveitamento dos recursos computacionais. Nesse cenário, o cyclic alcançou um Speedup escalado de 12,037× com 16 processos, enquanto o static apresentou apenas 4,736×. A partir desses resultados, é possível observar que o cyclic apresentou melhor escalabilidade nas configurações avaliadas. Apesar disso, ambas as estratégias permaneceram abaixo do desempenho ideal, principalmente devido aos custos de comunicação e ao desbalanceamento de carga.

Os principais gargalos identificados foram o desbalanceamento de carga na distribuição static, o custo de comunicação do MPI_Gather e a necessidade de reorganização dos dados no processo mestre na estratégia cyclic. Apesar de a distribuição cíclica ter reduzido significativamente o FBC, ela apresentou uma maior sobrecarga na coleta e reorganização dos dados, além de exigir maior utilização de memória. Outro fator limitante é que a implementação utiliza uma distribuição fixa de linhas entre os processos, sem a possibilidade de balanceamento dinâmico, além de exigir que a resolução da imagem seja divisível pelo número de processos. Por fim, os testes foram realizados em um único computador, não sendo possível avaliar os impactos da comunicação em rede entre diferentes máquinas, o que limita a análise do desempenho do MPI em ambientes distribuídos.

5. Conclusão

Neste projeto, foi possível compreender novas formas de abstração de hardware para a paralelização, principalmente por meio da utilização de processos e da possibilidade de execução em diferentes hardwares. Apesar de não ter sido possível utilizar mais de um computador para a execução com MPI, o projeto permitiu compreender diferentes formas de distribuição e organização do trabalho entre processos. Além disso, foi possível aprofundar o aprendizado sobre os custos de comunicação, a operação MPI_Gather e o balanceamento de carga, corrigindo conceitos e ampliando a compreensão sobre como esses fatores influenciam o desempenho e a eficiência da paralelização.

O MPI se mostrou ligeiramente superior à implementação com OpenMP, apresentando melhor desempenho em relação a fatores como o desbalanceamento de carga e a redução da

eficiência com o aumento do número de processos, considerando as diferentes estratégias de distribuição de trabalho avaliadas.

6. Referências

- KENDALL, Wes. Tutorials - MPI Tutorial. [S.l.], [s.d.]. Disponível em: https://mpitutorial.com/tutorials/. Acesso em: 29 out. 2026.
- OPEN MPI. Open Source High Performance Computing. [S.l.], [s.d.]. Disponível em: https://www.open-mpi.org/. Acesso em: 29 out. 2026.
- ORELLANA, Esbel Tomás Valero.. MPI - Scatter e Gather com o Filtro Gaussiano. Notas de aula da disciplina DEC107 Processamento Paralelo. UESC, 2026.
- TECHWITHCOSTA. Setup VS Code for WSL (Step by Step). YouTube, 21 ago. 2024. Disponível em: https://youtu.be/SyAlaukOOrQ?si=AUF3EpCOBw2qe70Y. Acesso em: 29 out. 2026.
- AMDAHL, Gene M. *Validity of the single processor approach to achieving large scale computing capabilities*. Disponível em: https://doi.org/10.1145/1465482.1465560. Acesso em: 21 set. 2026.
- GUSTAFSON, John L. *Reevaluating Amdahl's Law*. Disponível em: https://doi.org/10.1145/42411.42415. Acesso em: 21 set. 2026.

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Alecsander Augusto Paixão Almeida, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time)", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: Claude Sonnet 5, Chatgpt, Gemini 3.1 Pro, Gemini 3.8 Flash. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Revisão gramatical de trechos do relatório
- Auxílio na depuração (debugging) de blocos específicos de código
- Estruturação inicial de ideias e revisão bibliográfica
- Pesquisa de fontes
- Entender o problema e aprender o conteúdo relacionado a ele.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Itabuna - BA, 29/10/2026

Alecsander Augusto Paixão Almeida

Matrícula: 202311087

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, e Samuel Ferreira Santos, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "[Título do Trabalho]", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

( ) Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: ChatGPT 6.0

- Automação de processos de benchmarking
- Busca de fontes
- Revisão gramatical
- Debate de validade e argumentação do artigo
- Esclarecimento de pontos do trabalho e observações de lacunas

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus, 29/10/2026

Samuel Ferreira Santos

Matrícula: 202410518